Vidyalankar Institute of Technology

Department of Computer Engineering

DEVOPS MINI PROJECT

Week 11 Report

Automated Grocery Inventory Dashboard

Name:	Prachi Patil
Roll No.:	23102B0038
Class:	BE - VII
Division:	B
Academic Year:	2026–27

1. Objective

The objective of Week 11 was to containerize the Automated Grocery Inventory Dashboard using Docker and demonstrate the complete Docker image and container lifecycle.

The activity covered:

- Creating a Dockerfile for the Spring Boot WAR application.
- Building and tagging a Docker image.
- Creating and running a Docker container.
- Mapping host and container ports.
- Inspecting container details and application logs.
- Verifying the application through the browser.
- Demonstrating container stop, restart and removal.

2. Software and Technologies Used

Software / Technology	Purpose
Java 21	Application runtime
Spring Boot 4.1.0	Web application framework
Maven	Build and WAR packaging
MySQL	Application database
Apache Tomcat 10.1	Web application server
Docker 29.8.1	Containerization
Docker Compose 5.5.1	Container orchestration support
Git/GitHub	Source code management
Google Chrome	Application verification
VS Code	Project development and configuration
macOS	Development environment

3. Dockerfile Creation

The existing Spring Boot application was packaged as a WAR file using Maven. The WAR file generated was:

```
target/automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war
```

The Dockerfile uses Tomcat 10.1 with Java 21 and copies the generated WAR into the Tomcat `webapps` directory.

```
FROM tomcat:10.1-jdk21-temurin

RUN rm -rf /usr/local/tomcat/webapps/*
```

```
COPY target/automated-grocery-inventory-dashboard-0.0.1-SNAPSHOT.war \
     /usr/local/tomcat/webapps/automated-grocery-inventory-dashboard.war

EXPOSE 8080

CMD ["catalina.sh", "run"]
```

The container exposes Tomcat's internal port 8080.

4. Docker Image Creation

The WAR was first generated using:

```
./mvnw clean package -DskipTests
```

The build completed successfully and generated a 51 MB WAR file.
The Docker image was then created using:

```
docker build -t grocery-inventory-dashboard:v1.0 .
```

The image was successfully created with:

```
Repository: grocery-inventory-dashboard
Tag:        v1.0
Image ID:   5e3b1d110368
```

The image was verified using:

```
docker images
```

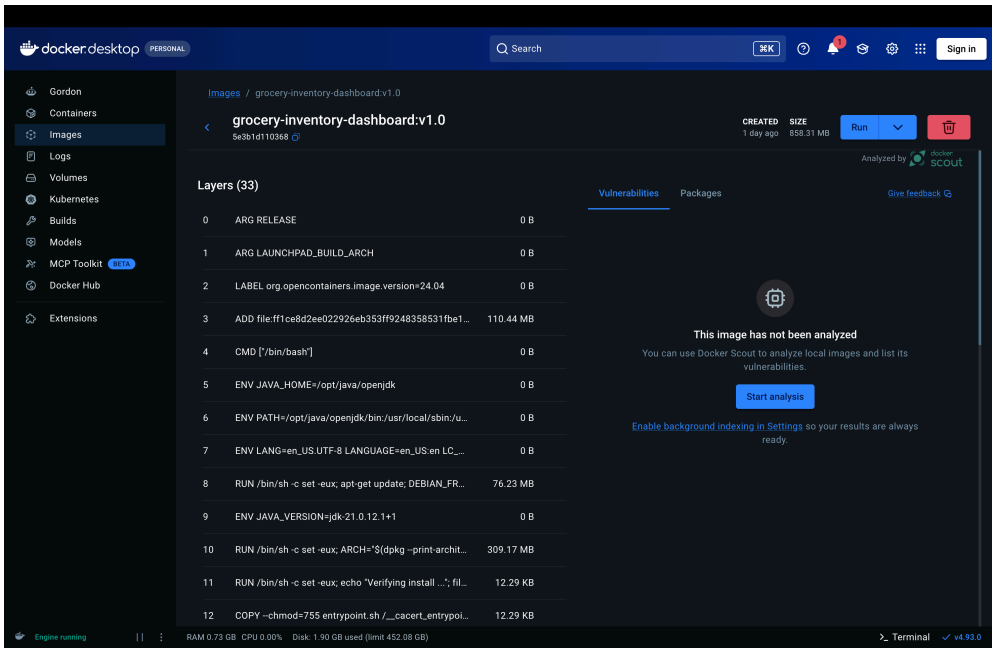


Figure 1: Docker image creation and image details

5. Container Creation and Port Mapping

The Docker container was created using:

```
docker run -d \
  --name grocery-inventory-container \
  -p 8083:8080 \
  -e SPRING_DATASOURCE_URL="jdbc:mysql://host.docker.internal:3306/grocer
  -e SPRING_DATASOURCE_USERNAME="root" \
  -e SPRING_DATASOURCE_PASSWORD="" \
  grocery-inventory-dashboard:v1.0
```

The port mapping was:

Host Port 8083 → Container Port 8080

Therefore, the application was accessed using:

```
http://localhost:8083/automated-grocery-inventory-dashboard/items
```

The application successfully loaded the Inventory Dashboard from the Docker container.

6. Database Connectivity

Initially, the container could not connect to MySQL because the application used `localhost:3306`. Inside a Docker container, `localhost` refers to the container itself.

The runtime database configuration was therefore changed to:

```
jdbc:mysql://host.docker.internal:3306/grocery_inventory
```

This allowed the containerized application to access the MySQL database running on the Mac host.

The Docker logs confirmed:

```
HikariPool-1 - Added connection
HikariPool-1 - Start completed.
Started AutomatedGroceryInventoryDashboardApplication
```

The WAR deployment and Tomcat startup then completed successfully.

7. Container Inspection and Logs

The running container was inspected using:

```
docker inspect grocery-inventory-container
```

Important verified values were:

```
Name:    /grocery-inventory-container
Status:  running
Image:   grocery-inventory-dashboard:v1.0
Port:    8083 -> 8080
```

A shorter inspection command was also used:

```
docker inspect --format='Name={{.Name}}
Status={{.State.Status}}
Image={{.Config.Image}}
Ports={{json .NetworkSettings.Ports}}' grocery-inventory-container
```

Container logs were obtained using:

```
docker logs grocery-inventory-container
```

The logs confirmed Tomcat startup, Spring Boot initialization, database connectivity and successful WAR deployment.

8. Container Lifecycle

The complete container lifecycle was demonstrated using Docker commands.

Operation	Command
Run container	`docker run -d --name grocery-inventory-container ...`
Check running containers	`docker ps`
Inspect container	`docker inspect grocery-inventory-container`
View logs	`docker logs grocery-inventory-container`
Stop container	`docker stop grocery-inventory-container`
Restart container	`docker start grocery-inventory-container`
Remove container	`docker rm grocery-inventory-container`
Verify removal	`docker ps -a`

The lifecycle can be represented as:

Image → Create → Running → Stop → Restart → Stop → Remove

9. Docker Command Log

The main commands executed during the activity were:

```
docker --version
docker compose version

./mvnw clean package -DskipTests
```

```
docker build -t grocery-inventory-dashboard:v1.0 .

docker images

docker run -d --name grocery-inventory-container \
-p 8083:8080 grocery-inventory-dashboard:v1.0

docker ps
docker logs grocery-inventory-container
docker inspect grocery-inventory-container

docker stop grocery-inventory-container
docker start grocery-inventory-container
docker rm grocery-inventory-container
docker ps -a
```

10. Final Result

The Docker implementation successfully demonstrated the required containerization workflow.

Requirement	**Status**
Dockerfile	Completed
Docker Image Build	Completed
Image Tagging	Completed
Container Creation	Completed
Port Mapping	Completed
Running Application	Completed
Container Logs	Completed
Container Inspection	Completed
Stop Container	Completed
Restart Container	Completed
Remove Container	Completed

11. Conclusion

Week 11 successfully containerized the Automated Grocery Inventory Dashboard using Docker.

A Tomcat-based Docker image was created using Java 21, the application WAR was deployed inside the container, and port 8083 on the host was mapped to Tomcat port 8080 inside the container.

The container was inspected through Docker commands, application logs were verified, and the application was successfully accessed through the browser. The container lifecycle was demonstrated through running, stopping, restarting and removing the container.

This completes the Docker image creation, container execution and container lifecycle requirements for Week 11.